Tutorial #7 - Skid Row

Introduction

Welcome to the seventh AppGameKit Studio tutorial. As before, these are aimed at coders who are just starting to explore the world of the AppGameKit Studio Scripting Tier1 coding language.

We previously created some basic 2D games and have worked our way through a range of coding methods and techniques, by now, if you've followed them all from the beginning, you'll have a fairly good understanding of coding and will have already delved deeper into game development by adding some of your own features.

For this tutorial we'll be expanding on what we've learned so far and taking a more in-depth look at creating some computer controlled AI cars!

The entry level for these tutorials expects some initial understanding of AppGameKit Studio and will reference similar coding practices used in our previous tutorials. For more beginner level tutorials, you can read the AppGameKit Studio Online Guides and watch the YouTube Getting Started Videos.

You are also encouraged to join the Official AppGameKit Studio forums. There's a new comers section where new users will be helped by many experienced and friendly developers who use AppGameKit Studio all the time.

Objectives

These tutorials aim to teach coding and game development using a combination of examples and action-based teaching. You will learn from studying the code snippets, reading and understanding the comments and gain an additional understanding of each section through a number of simple exercises aimed at reinforcing each lesson and will help build your confidence by demonstrating the ease with which the code can be altered.

During this tutorial you will:

- Learn how to make a very basic editor
- Develop more advanced AI
- Expand on a number of math techniques commonly used in game making
- Be introduced to some coding examples suitable for adding to a developer's library

At any point, examples of almost all coding commands can be accessed from the IDE (integrated development environment) simply by clicking directly to the left of the command and pressing F1 on the keyboard.

So, what game are we making today?

For this tutorial, we'll be using 2D sprites and expanding on what we've learned about basic AI. Some of you may know or remember the racing games of the late 1980's and early 1990's. It's this style of game we'll be recreating today.

In keeping with some of our latest tutorials, we'll assume that you've learned the basics and will only include limited comments in the code for areas we've covered in depth before. Of course, once again, if you have problems understanding anything, please refer to the previous tutorials for more information.

Like our previous tutorial, Meteor Dash, we'll be challenging you to take on some more complex coding and asking you to expand on the basic game.

First, we'll set up our project:

```
// Project: Skid Row
// Created: 2017-08-23

// show all errors
SetErrorMode(2)

// set window properties
SetWindowTitle( "Skid Row" )
SetWindowSize( 1024, 768, 1 )
SetWindowAllowResize( 1 ) // allow the user to resize the window

// set display properties
SetVirtualResolution( 1024, 768 ) // doesn't have to match the window
SetOrientationAllowed( 1, 1, 1, 1 ) // allow both portrait and landscape on mobile devices
SetSyncRate( 60, 0 )
UseNewDefaultFonts( 1 ) // since version 2.0.22 we can use nicer default fonts
```

To make the code easier to read, this project is split up into a number of sub files (.agc files). These

all need to be added to the main.agc via the #include command:

```
//let's include all of our sub files
#include "setupfunctions.agc"
#include "coreplayerfunctions.agc"
#include "coreaifunctions.agc"
#include "controlrace.agc"
#include "usefulmath.agc"
#include "mainmenu.agc"files.
```

In order, these include functions for various elements of the game:

setupfunctions.agc	This contains the functions needed to define the track, load sounds, set up text, position cars on the starting grid, etc.
coreplayerfunctions.agc	Not surprisingly, this contains the code that controls the player. It essentially only contains 2 functions, one to read the keyboard and apply the results to the player's car, and a second that displays information about the player.
coreaifunctions.agc	Our third file to include contains functions that control the AI cars. Here's you'll find routines that decide which direction a computer car will move, check for collisions, and a final function to check if the race is over.
controlrace.agc	Next in our list is a set of functions that control the race. These functions handle the basic drawing and updating of the race track and checks and displays car positions. Due to the nature of this, several functions are called to calculate everything correctly.
usefulmaths.agc	This file contains 3 important math functions, vital to the game, and ones that you'll find a need for time and time again. Here we find a function to wrap any angle to ensure it stays with 0 – 360 degrees. Another function to return the adjustments needed for one angle to be rotated to another at a set speed. Lastly, we've included a function that calculates the distance between 2 points in 2D space.
mainmenu.agc	Our last file contains the code for the main menu and game reset.

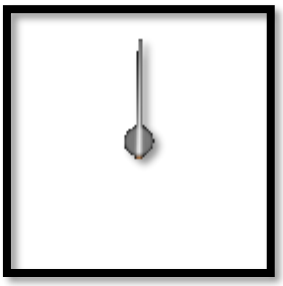
Okay, so, we've included our files, now let's load in some basic media and set up the game. The code is documented, but as these are areas that we've covered many times before, we won't go into great detail for most of it.

Let's start by looking at **main.agc**

```
//set up
//let's load a background

global background=50
loadimage(background,"background.png")

//we firstly need to set up a type for the track
type trackpieces

        //is the track in use
        active as integer
        //where will it be on screen
        x as float
        y as float
        //what kind of tile is it
        tracktile as integer
        //which sprite will it use.
        sprite as integer

endtype

//set up the sprites each track will use

dim tracks[225] as trackpieces

//special count
//each car counts the tracks it's hit during a lap,
//miss too many and the car is disqualified.

dim trackscounted[8,225]

//make track sprites
for makesprites=1 to 225
        tracks[makesprites].sprite=makesprites+99
        createsprite(tracks[makesprites].sprite,0)
next makesprites

//we will store all the tracks in an array
//and build the track from there
//this will allow for custom tracks.

//storage array for tracks,
//we will allow the track a depth of 15 units, (up/down)
//but it has an unlimited length
//for this demo we'll assume 15 as well as you'll see later
dim trackmap[15] as string

//load all of the track media
definetrackmedia()

//load the sounds we'll use
loadsounds()

//set up the text
setuptext()

//set up a type for the cars
type race
        x as float
        y as float
        //0 = not used
        //1 = player
        //>1 = ai
        active as integer
        sprite as integer
        image as integer
        speed as float
        angle as integer
        targetangle as integer
        rotatespeed as integer
        drift as float
        lap as integer
        total as integer
        text as integer
        position as integer
        travelled as integer
        warning as integer
```

```
        disqualified as integer
        finished as integer
endtype

//define an array
dim cars[8] as race

//global varibles
//store currentime
global currenttime:currenttime=GetMilliseconds()
//set up somewhere to store a timer to tidy the
//changes to the speed, otherwise it's too quick
global speedchange=0
//countdown to start
global startin
//has the race started
global raceon=0
//timer movement
Global frameadjust as float

//define start position
startwhere=setuptrack()

//load car media and place on track
definecarmedia(startwhere)

Global trafficlight=9997
LoadImage(trafficlight,"light.png")
CreateSprite(trafficlight,trafficlight)
SetSpriteAnimation(trafficlight,27.33,64,3)
SetSpriteVisible(trafficlight,0)
SetSpritePosition(trafficlight,700,300)

global speed=9998
LoadImage(speed,"speed.png")
CreateSprite(speed,speed)
SetSpritePosition(speed,8,8)
```

The needle is a little different to our other media as we want to rotate it around the centre so that it points correctly at the speedometer. For this to work, we've set the needle centre directly in the centre of the image, so that when we rotate the needle to the correct angle it will rotate around the centre circle.



```
//the needle is worth a mention as we want it to rotate around the centre.
global needle=9999
LoadImage(needle,"needle.png")
CreateSprite(needle,needle)
SetSpritePosition(needle,8,8)
SetSpriteAngle(needle,180)
```

Okay, so we're all set up. Let's look at what's next.

We will define a label to allow us to jump back to the code here to restart. A label must end with the colon character : Many programmers also prefix a label with a familiar symbol, in this case _. This makes labels easy to spot in blocks of code.

```
_startagain:
```

Now we'll jump to the main menu, so there is a pause between games. We'll take a look at that later.

```
mainmenu()
```

We are going to use a timer to count down the start of the race. Remember, TIMER() counts in seconds, you can also use GETMILLISECONDS() for a more precise count, but in this case we only need to know the nearest second.

```
startin=Timer()
```

Now let's look at the main game loop. If you've been following the tutorials from the beginning, there shouldn't be anything here that you don't recognise, so, we'll keep the explanations down to just the comments.

```
do

        //set up timers, including the frame adjustment for timer movement.
        frameadjust =60*GetFrameTime()
        currenttime=GetMilliseconds()

        //the time left is 3, minus the current timer, minus the value of startin
        //which we set at the start of the game.
        timeleft=3-round(timer()-startin)

        //if 3 seconds left, play engine start up sound
        if timeleft=3
                if GetSoundInstancePlaying(startupplaying)=0
                startupplaying=playsound(startup)
                endif
        endif

        //if 2 seconds left, play engines revving sound
        if timeleft=2
                if GetSoundInstancePlaying(startoffplaying)=0
                startoffplaying=playsound(startoff)
                endif
        endif


        //if less then 1 second, play the engine sound
        if getsoundinstanceplaying(engineplaying)=0 and timeleft<1
                engineplaying=playsound(engine)
        endif


        //if the race hasn't started (raceon)
        //and the timeleft>0 seconds
        //display the traffic light and set the correct animation frame
        //to cycle through the lights.
        If raceon=0 and timeleft>0
                SetSpriteVisible(trafficlight,1)
                SetSpriteFrame(trafficlight,timeleft)
        else
                //otherwise hide the traffic light sprite
                SetSpriteVisible(trafficlight,0)
                //and flag the race as started.
                raceon=1
        Endif


        //special for player.
        //if the player is warned more than 3 times for being off track
        //the race ends as the player will be flagged as disqualified.
        if cars[1].disqualified<>0
                //set all the cars as having finished the race
                For endrace=1 to 8
                        //set the laps completed to 4.
                        cars[endrace].lap=4
                Next endrace
        endif

        //check to see if the race is over

        isitdone=checkfinalpositions()
```

```
        //find out how many cars have been disqualified

        howmany=8

        //count them down
        For countthem=1 to 8
                if cars[countthem].disqualified<>0
                        dec howmany,1
                endif
        next countthem

        //if all the cars are flagged as finished that are still in the race or the player
has finished
        If isitdone>=howmany or cars[1].finished<>0
                //jump back to the main menu
                goto _startagain
        endif

    //draw the track
    drawtrack()

    //has the race started?
    //if so, control the cars
    if raceon=1
    controlcars()

    //if the race isn't over,
    //check the player controls
    if isitdone<>8
    controlplayer()
    endif

    //find the cars current position
    findposition()


    endif

    //show the player stats
    playerstats()

        //update the screen
    Sync()
loop
```

Let's break this down into some smaller sections and take a look at:

controlrace.agc
These functions control any race feature that doesn't directly involve controlling the player or the AI controlled cars.

Click on the above file in the right-hand window of your IDE, and we'll take a look at this in detail.

`Drawtrack()` does just that. It places the appropriate track sprite at the correct position. We will create the appearance of movement later by adjusting the track position.

```
//draw the track
function drawtrack()

        //run through all elements of the track
        For draw=1 to 225
                //if they are active
                if tracks[draw].active<>0
                        //set the track at the position stored in tracks[?].x and track[?].y
                        SetSpritePosition(tracks[draw].sprite,tracks[draw].x,tracks[draw].y)
                endif
                //keep going
        next draw

endfunction
```

The next function in our sub file is `displaypositions()`. You won't be surprised to know that this displays the car positions. In our game, this is only called after a race, but could just as easily be called every frame to give live position updates, or even create a commentary with just a few minor code changes. Once again, we won't bore you with long explanations as this should all be familiar code by now, just read the comments and study the code used:

```
Function displaypositions()

        //calculate the position
        //we'll use a basic sort for this
        //if one car came higher than another
        //display those stats first.

        for whichcar=1 to 8
                for where=1 to 8
                        //if car (whichcar]'s position is equal to where
                        //display this cars position
                        if cars[whichcar].position=where
                        SetTextSize(cars[whichcar].text,32)
                        //set A$ to show the position
                        a$=GetTextString(cars[whichcar].text)
                        //store the final result in B$
                        b$="Car "+str(whichcar)+" is "+a$
                        //set the string
                        SetTextString(cars[whichcar].text,b$)
                        //place the appropraite car sprite
                        SetSpritePosition(cars[whichcar].sprite,400,120+(where*48))
                        //rotate to face right
                        SetSpriteAngle(cars[whichcar].sprite,90)
                        //set the text position
                        SetTextPosition(cars[whichcar].text,500,130+(where*48))
                        endif
                        //keep going if more to do.
                next where
        next whichcar

EndFunction
```

Next in our list is `updatetrack(x,y)`. Once again, this should be familiar to you. We enter the function passing the amount we want to move the track in the x and y positions. We apply this to the tracks, and then the AI cars (as they can be moved along with the scrolling screen). You'll see a little later how we calculate this movement.

```
Function updatetrack(x as float,y as float)

        //scan all tracks
        for updatetrack=1 to 225

                //update the position by x and y
                tracks[updatetrack].x=tracks[updatetrack].x+x
                tracks[updatetrack].y=tracks[updatetrack].y+y
                //carry on if more to do
        next updatetrack

        //do the same for all the AI controlled cars
        for updatecars=2 to 8

                cars[updatecars].x=cars[updatecars].x+x
                cars[updatecars].y=cars[updatecars].y+y

                //check to see if the car hits another
                checkcarcrashes(updatecars)
                //carry on if more to do
        next updatecars

EndFunction
```

Next we will count the tracks. We adjust each car individually and pass the car we want to check into the function using the value **use**. This function checks to see if the car has hit a track and then flags it has hit using the array `trackscounted(car,track)`. We flag the track so it can only be

counted once per lap. Although there's nothing really new being shown here, this routine will benefit from a little more explanation. Read through the code below and we'll flesh out some of the details.

```
Function counttracks(use)

        //set some variables
        total=0
        laphit=0
        totalactive=0

        //loop through all tracks
        For count=1 to 225
        //look at all tracks
         If tracks[count].active<>0
                //count all the appropriate tracks (not grass or trees)
                if tracks[count].active<14
               inc totalactive,1
               endif
```

As you will have noticed, we are counting tracks with a value of less than (<) 14. This means it's a valid track. This will automatically count the total number of tracks in our race that need to be driven to complete our circuit. By counting the tracks in this way, we can allow for many track variations, with different numbers of active tracks.

We now will check to see if the car has collided with a sprite.

```
        If GetSpriteCollision(cars[use].sprite,tracks[count].sprite)
```

If the track hit has a value of 9 assigned to it, it is the starting grid, and flag the lap as being hit.

```
               //track 9 is the starting grid
               If tracks[count].active=9
                      laphit=1
               Endif
```

If the car hasn't hit this track before, we will flag it. If it's a track, (<14) we flag it with the value of 1. If it's not a track, we flag it with a value of 2. We will use this later in an exercise. You'll notice that if it's not a track, we also increase the cars warnings. If warnings rise above 3 the car is flagged as disqualified and can't win. In practice this only applies to the player, as the AI won't leave the track unless forced off. If you read down, you'll also notice that cars off the track slow down.

```
               //flag the track has driven over
               If trackscounted[use,count]=0
                      //is it track?
                      if tracks[count].active<14
                      trackscounted[use,count]=1
                      //increase the tracks the car has travelled.
                      inc cars[use].travelled,1
                      endif

                      //is the car off the track?
                      if tracks[count].active>=14
                             trackscounted[use,count]=2
                             //flag the offending car as having left the track
                             inc cars[use].warning,1
                             //if the car has missed 3 tracks this lap
                             //it is out of the race.
                             if cars[use].warning>3
                                    cars[use].disqualified=1
                             endif

                             //if the car isn't on the track,
                             //slow it down.
                             if cars[use].speed>.1
                                    dec cars[use].speed,.1
                             endif
```

```
                                //the car IS off the track, so show a warning,
                                //in practice this only applies to the player,
                                //as the AI will never leave the track in our tutorial.
                                SetTextString(1,"Off Track WARNING:"+str(cars[use].warning))
                        endif


                Endif
        Endif
 Endif
Next count
```

Now that we have counted all of our tracks, we run through them all again, and count the ones that count as track and have been hit. We then store the current number of tracks hit this lap in the `cars[use].total` for later reference.

```
//set the total number of tracks hit this race by counting them

For add=1 to 225
        if trackscounted[use,add]=1
                inc total,1
        endif
Next add

cars[use].total=total
```

If the car has hit the lap, but hasn't hit all the tracks, the lap doesn't count. Again for this demo, this really only applies to the player. As hitting a lap is a special event, we change the total to 999, which means lap completed (Note: We allow three tracks to be missed.).

```
//check if starting grid hit,
//give the player a change to miss a couple of tracks
if laphit=1
        if cars[use].total>=(totalactive-3)
                total=999
                //if the player has missed more than 3 tracks it's game over
                else
                //flag total as 1000, which means the car loses
                total=1000
        endif

endif

//if the total is 999, the car has completed the lap
```

So, the car has completed the lap. We now run through all the tracks and flag them as uncounted, ready for the next lap. We also clear the cars warnings as it's a new lap. Lastly, we increase the number of laps the car has completed, reset total to 0, and play some cheerful clapping.

```
        if total=999
                        //reset all of the tracks hit to 0
                for reset=1 to 225
                        trackscounted[use,reset]=0
                next reset

                //set the cars warning to 0
                        cars[use].warning=0
                //make total 0
                //and add to the laps the completed
                total=0
                inc cars[use].lap,1

                if GetSoundInstancePlaying(clappingplaying)=0
                        clappingplaying=playsound(clapping)
                endif

        endif

        //jump back, returning the total laps

EndFunction total
```

Now we reach another function. Again, this code doesn't contain anything you don't already know, but this is where we calculate the position of all of the cars and swap them around if needed. Read the comments for a full explanation.

```
Function findposition()
	//run through all cars and compare them with the other cars.
	for test=1 to 8
		for check=1 to 8
			//find out how far each has travelled.
			store1=cars[check].travelled
			store2=cars[test].travelled

			//if the test car (store2) has travelled further than the check car
			(store1)
			//and the test car is in a lower position than the check car
			if store2>store1 and cars[test].position>cars[check].position`

				//store the positions for reference.
				temp=cars[check].position
				temp2=cars[test].position

				//swap the positions over
				if cars[check].finished=0
				cars[check].position=temp2
				endif

				if cars[test].finished=0
				cars[test].position=temp
				endif

				//see if player is near enough to hear
				dist=QuickDistance(cars[1].x,cars[1].y,cars[test].x,cars[test].y)
				if dist<600
					//play the overtaking sound
				PlaySound(overtake)
				endif

			endif

		next check
	next test

	//apply a nice st,nd,rd or th to the string
	//i.e. 1st, 2nd, 3rd, 4th, etc.
	for apply=1 to 8
		t1=cars[apply].position
		a$=str(cars[apply].position)
		b$="th"
		if t1=1 then b$="st"
		if t1=2 then b$="nd"
		if t1=3 then b$="rd"

		a$=a$+b$
		//set the cars position string with the new position
		settextstring(cars[apply].text,a$)
	next apply
EndFunction
```

So, we've drawn and updated our tracks, and worked out which cars are in which position. The last function of this sub-file is `checktrack(?)`. We pass the car we want to check and test it for collisions against each track sprite. We need to know which track a car is on so our AI can correctly navigate around them.

```
Function checktrack(which)

	done=0

	For checking=1 to 225
		If tracks[checking].active<>0
			If GetSpriteCollision(tracks[checking].sprite,cars[which].sprite)
				done=checking
				checking=226
			Endif
```

```
                EndIf
        Next checking

EndFunction done
```

We haven't commented on this code as if you've been keeping up, everything should be self-explanatory, but remember, if you find you don't understand, refer to earlier examples for fuller details.

Now that we've looked at controlling our tracks, we'll need some cars to drive around it. We'll start with a breakdown of the code controlling the AI cars.

Let's look at `controlcars()`, this is where all the AI magic happens

```
Function controlcars()
//run through all 8 cars
        for controlcars=1 to 8
        //only bother if the car is active.
        if cars[controlcars].active<>0
        //count the tracks the car has hit
        counttracks(controlcars)
        //if the car hasn't finished, display position next to the car
        If cars[controlcars].finished=0

        SetTextPosition(cars[controlcars].text,cars[controlcars].x+sin(cars[controlcars].angl
e)*48,cars[controlcars].y+cos(cars[controlcars].angle)*64)

        endif

        //set the sprite angle for the car
        //we're using one of our useful math functions here to ensure that the sprite angle
        //remain in the rang
        cars[controlcars].angle=wrapangle(cars[controlcars].angle)
        SetSpriteAngle(cars[controlcars].sprite,cars[controlcars].angle)
        //if it's car 1, which is the players car, adjust by speed
        if controlcars=1
                movecarx#=-sin(cars[1].angle)*cars[1].speed*frameadjust
                movecary#=cos(cars[1].angle)*cars[1].speed*frameadjust
        //if not the players car, it's an AI, so adjust, but by slightly faster to add a
challange.
                Else

        cars[controlcars].x=cars[controlcars].x+sin(cars[controlcars].angle)*(cars[controlcar
s].speed*1.2*frameadjust)
        cars[controlcars].y=cars[controlcars].y-
cos(cars[controlcars].angle)*(cars[controlcars].speed*1.2*frameadjust)

        //set a sprite position
        SetSpritePosition(cars[controlcars].sprite,cars[controlcars].x,cars[controlcars].y)
        endif

        //find out which track the car is on
        whatisthecaron=checktrack(controlcars)

        //if it's not car 1 (player car), find out the image ID of the track
        If whatisthecaron<>0 and controlcars<>1
                whattile=GetSpriteImageID(tracks[whatisthecaron].sprite)

                //drift if not player car for a little realism,
                if controlcars<>1
                aa=cars[controlcars].angle+RandomSign(90)
                cars[controlcars].x=cars[controlcars].x+sin(aa)*2*frameadjust
                cars[controlcars].y=cars[controlcars].y-cos(aa)*2*frameadjust
                //speed

                //if not a corner, increase AI car speed
                if whattile>4 and whattile<10 and cars[controlcars].speed<10 and
random(1,999)>800
                        inc cars[controlcars].speed,.4
                endif

                //slow AI cars on corners.
                if whattile<5 or whattile>9 and cars[controlcars].speed>7
                        dec cars[controlcars].speed,.1
```

```
endif

Endif

//Not we need to control the AI cars.
```

Now let's look at the AI. As with our previous tutorial, the AI is only pseudo (ie. it's being faked). This is actually true of almost all game related AI, as what we aim to achieve is the appearance of intelligence rather than a learning machine.

To give this effect, we will adjust a car's movements to match the track to give the appearance of driving. For example, we know that if we are on a straight the cars need to drive at a certain angle, or if the car is turning a corner, we'll need to adjust the sprite to another angle. We do this slowly, so the cars appear to be driving.

In our `usefulmath()` function, we have a number of clever little routines which do a lot of the work for us. One of the most important for our racing game is `CurveAngle()`. To this we pass a number of values.

CurveAngle(target angle, current angle, speed of turn)
This returns a value, that provides a new angle that needs to be applied to the sprite. This moves the sprite rotation closer to the target angle at increments of "speed of turn". We apply this each turn to move towards our target angle.

As you'll remember, we stored the current track ID in "whattile". Now let's decide what the AI needs to do with it.

Let's take ID 1 as an example. The car needs to turn to 180 degrees.

We pass 180 as our target angle, as well as the current angle of the sprite for the car to `CurveAngle()`. For corners we calculate the speed of turn based on: X amount divided by the cars speed, so that the car slowly turns into the corner. This value varies for each corner to achieve the maximum control.

```
Select whattile

Case 1
cars[controlcars].rotatespeed=CurveAngle(180,cars[controlcars].angle,180/cars[controlcars].speed)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 2
carr[controlcars].rotatespeed=CurveAngle(270,cars[controlcars].angle,180/cars[controlcars].speed)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 3
cars[controlcars].rotatespeed=CurveAngle(0,cars[controlcars].angle,180/cars[controlcars].speed)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 4
cars[controlcars].rotatespeed=CurveAngle(90,cars[controlcars].angle,90/cars[controlcars].speed)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase
```

```
Case 5
cars[controlcars].rotatespeed=CurveAngle(90,cars[controlcars].angle,2)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 6
cars[controlcars].rotatespeed=CurveAngle(180,cars[controlcars].angle,2)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 7
cars[controlcars].rotatespeed=CurveAngle(0,cars[controlcars].angle,2)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 8
cars[controlcars].rotatespeed=CurveAngle(270,cars[controlcars].angle,2)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 10
cars[controlcars].rotatespeed=CurveAngle(0,cars[controlcars].angle,180/cars[controlcars].speed
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case
11cars[controlcars].rotatespeed=CurveAngle(270,cars[controlcars].angle,180/cars[controlcars].spee
d)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 12
cars[controlcars].rotatespeed=CurveAngle(270,cars[controlcars].angle,180/cars[controlcars].speed)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

Case 13
cars[controlcars].rotatespeed=CurveAngle(90,cars[controlcars].angle,360/cars[controlcars].speed)
cars[controlcars].angle=cars[controlcars].rotatespeed

EndCase

EndSelect

Endif

Endif

//now we send the movement values to the update track

updatetrack(movecarx#*frameadjust,movecary#*frameadjust)

//keep going if there are more cars
next controlcars

EndFunction
```

The next two functions here, we'll leave you with just the comments, as once again, we've covered this kind of coding before.

```
//cars car collide
Function checkcarcrashes(this)

        //compare all cars to check for collisions
        For check=1 to 8
                //only if the cars aren't the same
                if check<>this
                        //if they've hit
                        If GetSpriteCollision(cars[check].sprite,cars[this].sprite)
                        //calculate the angle between the 2 cars
```

```
                        adjx#=cars[check].x-cars[this].x
                        adjy#=cars[check].y-cars[this].y

                        tangle=ATanFull(adjx#,adjy#)

                        //bounce them off of each other
                        if check<>1
                        cars[check].x=cars[check].x+sin(tangle)*1
                        cars[check].y=cars[check].y-cos(tangle)*1
                        endif

                        if this<>1
                        cars[this].x=cars[this].x-sin(tangle)*1
                        cars[this].y=cars[this].y+cos(tangle)*1
                        endif

                         endif
                Endif

        //keep going if more.
        Next check

EndFunction

//once the race is over
Function checkfinalpositions()
        //set the race as not over
        istheraceover=0
        //check all the cars
        For checking=1 to 8
                //if a car has completed 3 laps and isn't flagged as finished
                If cars[checking].lap>=3 and cars[checking].finished=0
                        //set the finished flag to the cars current position
                        cars[checking].finished=cars[checking].position
                endif
        Next checking

        //see if all cars have finished
        For count=1 to 8
                if cars[count].finished<>0
                        inc istheraceover,1
                endif
        Next count

        //return the number of finished cars

EndFunction istheraceover
```

Now we'll take a quick look at the player controls. Again, this is code that we've cover many times. We check key presses, adjust variables, show and hide text, etc. Read the comments for more details.

```
Function controlplayer()

        //jump out
        if GetRawKeyState(27)
                end
        Endif


        //left
        If GetRawKeyState(37)
                dec cars[1].angle,4

                if cars[1].speed>.1
                dec cars[1].speed,.03
                endif

        Endif

        //right
        If GetRawKeyState(39)
                inc cars[1].angle,4

                if cars[1].speed>.1
                dec cars[1].speed,.03
                endif

        Endif
```

```
        //speed up
        If GetRawKeyState(38) and cars[1].speed<1.5
                inc cars[1].speed,.02
        endif

        if cars[1].speed>.01 and GetRawKeyState(38)=0
        dec cars[1].speed,.015
        endif

        //brake
        If GetRawKeyState(40) and cars[1].speed>0
                dec cars[1].speed,.4
        endif

EndFunction

Function playerstats()

        //set the MPH
        //delay to give better looking display

        ts=cars[1].speed*175

        ts=round(ts*5)/5

        SetSpriteAngle(needle,180+ts)

        SetTextString(2,GetTextString(cars[1].text))

        SetTextString(3,"LAP "+str(cars[1].lap)+"/3")

        //only display the warning if it's the player car

        if cars[1].warning>0
                SetTextVisible(1,1)
                else
                SetTextVisible(1,0)
        endif

EndFunction
```

Okay, so we're moving forward with our coding. Once again, this should all be familiar, so for `mainmenu()`, we'll leave you with just the comments.

```
Function mainmenu()

        //hide lap and position text
        SetTextVisible(2,0)
        SetTextVisible(3,0)
        //show title text
        SetTextVisible(4,1)
        SetTextVisible(5,1)

        //if the player car has finished
        //the game is over
        if cars[1].finished<>0
        SetTextString(1,"THE RACE IS OVER")
        displaypositions()
        endif

        //the race has finished
        raceon=0

        //start a loop
        do

        //if the player presses enter, jump out.
        If GetRawKeyState(13)
                exit
        endif

        //update the menu
        Sync()

        //repeat the loop
        loop
```

```
        //make lap and positon visible
        SetTextVisible(2,1)
        SetTextVisible(3,1)
        //hide title text
        SetTextVisible(4,0)
        SetTextVisible(5,0)

        //reset all the car data
        clearcars()

        //reset all the track data.
        cleartrack()

        //build the track,
        //returning the start position
        startwhere=setuptrack()

        //run through all 8 cars
        //and position them
        For make=1 to 8
                positioncars(make,startwhere)
        next make

        //draw the track
        drawtrack()

EndFunction

//set all track pieces to not active.
Function cleartrack()

        For clearing=1 to 225
                tracks[clearing].active=0
        Next clearing

EndFunction
```

`Clearcars()` should also be pretty clear. Here we run through all cars, including the player cars and reset the values to 0. We do this before every race.

```
Function clearcars()

        //run through all 8 cars and reset important data.
        for clearcars=1 to 8
                //reset has car been disqualified
                cars[clearcars].disqualified=0
                //reset has car finished race
                cars[clearcars].finished=0
                //reset distance travelled to 0
                cars[clearcars].travelled=0
                //reset the cars position
                cars[clearcars].position=0
                //reset car off track warning
                cars[clearcars].warning=0
                //reset the text that displays the cars position to blank ""
                SetTextString(cars[clearcars].text,"")
        next clearcars

Endfunction
```

Now let's look at the `setupfunctions.agc` sub-files. Here, as the name suggests, we set up our game, including loading media and building our track. Let's load in our track media:

```
Function definetrackmedia()

        global track1=1:loadimage(track1,"track1.png")
        global track2=2:loadimage(track2,"track2.png")
        global track3=3:loadimage(track3,"track3.png")
        global track4=4:loadimage(track4,"track4.png")
        global track5=5:loadimage(track5,"track5.png")
        global track6=6:loadimage(track6,"track6.png")
        global track7=7:loadimage(track7,"track7.png")
        global track8=8:loadimage(track8,"track8.png")
        global track9=9:loadimage(track9,"track9.png")
```

```
        global track10=10:loadimage(track10,"track4.png")
        global track11=11:loadimage(track11,"track3.png")
        global track12=12:loadimage(track12,"track2.png")
        global track13=13:loadimage(track13,"track1.png")
        global trees=14:loadimage(trees,"trees.png")

        `0 = no track
        `0
        `1 = corner left and down
        `A
        `2 = corner right and down
        `B
        `3 = corner right and up
        `C
        `4 = corner left and up
        `D
        `5 = bottom straight
        `E
        `6 = left straight
        `F
        `7 = right straight
        `G
        `8 = top straight
        `H
        `9 = bottom straight, starting grid
        `I
        `10=corner, left and up
        `J
        `11=Cornet, right and up
        `K
        `12=corner, up and left
        `L
        `13=corner up and right
        `M
        `14=Trees
        `N
EndFunction
```

`Setuptrack` is an example of a simple level editor. Here we read strings, extract the data and convert it into a track. Read through the comments for details.

```
        Function setuptrack()

                //we can build our tracks here.
                //take a look at the values above. We use a letter to represent different
                //types of tracks
                //of example, track 1 is represented by A, track 4 is represented by D
                //you can design your own tracks by editing this data.


                trackmap[1] ="N00N0AHHHB000N0"
                trackmap[2] ="00000F00NG00000"
                trackmap[3] ="00000FN00G0N000"
                trackmap[4] ="0AHHHK000JHHHB0"
                trackmap[5] ="0F0000N000000G0"
                trackmap[6] ="0F0N0000000NMC0"
                trackmap[7] ="0FN00000000NG00"
                trackmap[8] ="0FN000000000GN0"
                trackmap[9] ="0F00000000N0JLN"
                trackmap[10]="0FN0000N00000G0"
                trackmap[11]="0F00N0N0N00N0G0"
                trackmap[12]="0DEEEEIEEEEEEC0"
                trackmap[13]="000N0000NNNN000"
                trackmap[14]="00000N0000N0000"
                trackmap[15]="000000000000000"


                //this is the start position
                startx=0
                starty=0
                startgrid=0

                //build map
                //scane the map
                for down=1 to 15
                        //run across string, using the string length for the width of
                        //the track
```

```
	for across=1 to len(trackmap[down])
		//we are using some of the string commands here
		//in this exampl, MID() reads the middle of a string
		//press F1 for more information
		what$=mid(trackmap[down],across,1)
		//if it's 0 then do nothing
		If what$<>"0"
		//otherwise uss ASC() to get the numeric value of the string
		//subtract 64 from it (because the numeric value of A is 65)
		//which will leave us with the number, which ties in with the
		//image ID of the track
		tracktype=asc(what$)-64
		endif

		//if 0, make track tracktype 99
		if what$="0" then tracktype=99

		//find a free piece of track
		use=findfreetrack()
		//if avalable
		if use<>0
			//set the track active as the track type
			tracks[use].active=tracktype
			//if 99 use the background image
			if tracktype=99 then tracktype=background
			//set the image
			SetSpriteImage(tracks[use].sprite,tracktype)
			//set the sprite collision shape
			SetSpriteShape(tracks[use].sprite,3)
			//set the size
			SetSpriteSize(tracks[use].sprite,514,514)
			//set the track position
			tracks[use].x=startx
			tracks[use].y=starty


			//if this is the start grid, make a note of it.
			if tracktype=9
				startgrid=use
			endif

		endif

		//adjust the position of the next track, and carry on.
		inc startx,512
	next across
		startx=0
		inc starty,512
	next down


	//once built, set screen focus on start grid.

	//find the first active track
	//we need this for first positioning.
	thefirsttrack=0
	for look=1 to 225
		if tracks[look].active<>0
			thefirsttrack=look
			look=226
		endif
	next look

	offsetx=0
	offsety=0

	//find the difference between the first track tile with something in and the
startgrid

	if thefirsttrack<>0
		if startgrid<>0
			offsetx=256+(tracks[thefirsttrack].x-tracks[startgrid].x)
			offsety=128+(tracks[thefirsttrack].y-tracks[startgrid].y)
		endif

	endif

	//offset all active track tiles by that amount
	//and set the tracks initial position
	for doit=1 to 225
```

```
                if tracks[doit].active<>0
                        tracks[doit].x=tracks[doit].x+offsetx
                        tracks[doit].y=tracks[doit].y+offsety
                        SetSpritePosition(tracks[doit].sprite,tracks[doit].x,tracks[doit].y)
                endif
        next doit

        //return the starting grid tile.

Endfunction startgrid
```

EXERCISE: *Design your own map, using a 15x15 template. Use the following Letters for each track type. In our demo, the starting grid MUST point left to right:*

0 = no track
A = corner left and down
B = corner right and down
C= corner right and up
D = corner left and up
E = bottom straight
F = left straight
G= right straight
H= top straight
I = bottom straight, starting grid
J=corner, left and up
K=Cornet, right and up
L=corner, up and left
M=corner up and right
N=Trees

The last few functions here load in media and place cars on the starting grid, offset from the starting grid position. Once again, this should need little explanation, so we'll stick with the comments. We've deliberately left these sparse here as you should by now have a good understanding of what is going on.

```
Function definecarmedia(starttrack)

global car1=20:loadimage(car1,"car1.png")
global car2=21:loadimage(car2,"car2.png")
global car3=22:loadimage(car3,"car3.png")
global car4=23:loadimage(car4,"car4.png")
global car5=24:loadimage(car5,"car5.png")
global car6=25:loadimage(car6,"car6.png")
global car7=26:loadimage(car7,"car7.png")
global car8=27:loadimage(car8,"car8.png")

for makecars=1 to 8
        cars[makecars].sprite=999+makecars
        cars[makecars].text=999+makecars
        cars[makecars].angle=90
        CreateSprite(cars[makecars].sprite,19+makecars)
        SetSpriteDepth(cars[makecars].sprite,8)
        SetSpriteAngle(cars[makecars].sprite,cars[makecars].angle)
        CreateText(cars[makecars].text,"")
        SetTextSize(cars[makecars].text,24)
        cars[makecars].active=makecars
        cars[makecars].speed=0
        cars[makecars].position=makecars

        positioncars(makecars,starttrack)

next makecars

EndFunction
```

```
Function setuptext()

        //show warning

        CreateText(1,"")
        SetTextSize(1,48)
        SetTextPosition(1,300,600)
        SetTextColor(1,127,0,0,255)

        //show position
        CreateText(2,"")
        SetTextSize(2,96)
        SetTextPosition(2,880,16)

        //lap
        CreateText(3,"LAP")
        SetTextSize(3,64)
        SetTextPosition(3,400,16)

        //title
        CreateText(4,"SKID ROW")
        SetTextSize(4,96)
        SetTextPosition(4,330,80)
        SetTextVisible(4,0)

        //start race
        CreateText(5,"PRESS ENTER")
        SetTextSize(5,96)
        SetTextPosition(5,300,650)
        SetTextVisible(5,0)

EndFunction


Function findfreetrack()

        thisoneisfree=0

        for search=1 to 225
                //hide the sprite just in case it's not going to be used.
                SetSpritePosition(tracks[search].sprite,-100000,-10000)
                if tracks[search].active=0
                        thisoneisfree=search
                        search=226
                endif
        next search

Endfunction thisoneisfree

Function positioncars(makecars,starttrack)

        //set the car positions based on the position of the starting grid.
        //all cars are offset based on the starting grid.
        select makecars

                case 1

                x1=tracks[starttrack].x+256
                y1=tracks[starttrack].y+64

                endcase

                case 2

                x1=tracks[starttrack].x+256
                y1=tracks[starttrack].y+160

                endcase

                case 3

                x1=tracks[starttrack].x+256
                y1=tracks[starttrack].y+256


                endcase

                case 4
```

```
                x1=tracks[starttrack].x+256
                y1=tracks[starttrack].y+352

                endcase

                case 5

                x1=tracks[starttrack].x+128
                y1=tracks[starttrack].y+64


                endcase

                case 6

                x1=tracks[starttrack].x+128
                y1=tracks[starttrack].y+160

                endcase

                case 7

                x1=tracks[starttrack].x+128
                y1=tracks[starttrack].y+256


                endcase

                case 8

                x1=tracks[starttrack].x+128
                y1=tracks[starttrack].y+352


                endcase

        endselect

        //pass the postion to the x and y variables
        cars[makecars].x=x1
        cars[makecars].y=y1
        //reset the car stats
        cars[makecars].angle=90
        cars[makecars].disqualified=0
        cars[makecars].warning=0
        cars[makecars].travelled=0
        cars[makecars].lap=0
        cars[makecars].position=makecars

        //set the cars sprite position
        SetSpritePosition(cars[makecars].sprite,x1,y1)
        //reset angle
        SetSpriteAngle(cars[makecars].sprite,cars[makecars].angle)

EndFunction

Function loadsounds()

        global engine:engine=loadsound("engine.wav")
        global engineplaying=0
        global overtake:overtake=loadsound("overtake.wav")
        global startoff:startoff=loadsound("startoff.wav")
        global startoffplaying=0
        global startup:startup=loadsound("startup.wav")
        global startupplaying=0
        global clapping:clapping=loadsound("clapping.wav")
        global clappingplaying=0

EndFunction
```

Our last sub-file contains some useful math. These tutorials don't aim to teach mathematics, so we'll leave that for you to investigate further. Suffice to say, these three functions are some of the most useful you'll find, and have many, many uses in a wide range of games.

```
//this command will return a value between 0 and 360 degrees
//feel free to use this in your own projects.
```

```
function wrapangle(angle as float)
    getout=0
    jumpout=0
    getout=angle
    getout=getout-mod(getout,360)
    angle=angle-getout
       jumpout=9999

    while angle<0.0 or angle>=360.0
        if angle<0.0 then angle=angle+360.0
        if angle >=360.0 then angle=angle-360.0
        dec jumpout,1
        if  jumpout=0
         exit
       endif

    endwhile
    if  jumpout=0 then angle=0.0

endfunction angle


//This command will return an auto-interpolated angle based at a given speed.

function CurveAngle( destination as float, current as float,  speed as float)
    local diff as float
    if speed < 1.0 then speed = 1.0
    destination = WrapAngle( destination )
    current = WrapAngle( current )
    diff = destination - current
    if diff <- 180.0 then diff = ( destination + 360.0 ) - current
    if diff > 180.0 then diff = destination - ( current + 360.0 )
    current = current + ( diff / speed )
    current = WrapAngle( current )
endfunction current


//Calculate the distance between 2 points in 2D space.

Function QuickDistance(X1#,Y1#,X2#,Y2#)

  distance#=0

  Dist_X# = X1# - X2#
  Dist_Y# = Y1# - Y2#

 Distance# = Sqrt(Dist_X# * Dist_X# + Dist_Y# * Dist_Y#)

 EndFunction distance#
```

Well, now you have a complete breakdown of the code. Hopefully this has been clear enough for you to make some of your own changes. There are a lot of features you might like to add and we're sure you can think of some great ideas, we'll give you one example to try adding for yourself.

EXERCISE: *Using everything you've learned so far, attempt to add oil slicks to your tracks that slow cars. We're leaving this one totally up to you, but here are some clues.*

1. You'll need to source or draw an an oil slick that is 64x64 pixels in width and height
2. The slicks will need to move as the screen scrolls. You can find clue on this by studying the function updatetrack(), and drawtrack()
3. Cars will need to lose speed when they hit a slick. You'll need to check collisions for this

Give that a go, we're confident that this task is well within your abilities, and if you've followed all of our tutorials to date you should find this one quite easy. Once again, if you have any problems, don't be afraid to reference previous tutorials.